Nama : *Rafika Lanan Soge*

Nim : *257111057*

Kelas/Prodi : *C/Pendidikan Informatika*

Mata kuliah : *Struktur Data*

Analisis Kompleksitas Single Linked List

```
class Mahasiswa:
    def __init__(self, nim, nama, jurusan):
        self.nim = nim
        self.nama = nama
        self.jurusan = jurusan

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None # Hapus baris ini kalau mau jadi Single Linked List

class LinkedListMahasiswa:
    def __init__(self, double=True):
        self.head = None
        self.tail = None
        self.double = double # True = Double, False = Single

    # 1. TAMBAH DI AWAL
    def tambah_awal(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            node.next = self.head
            if self.double:
                self.head.prev = node
            self.head = node
        print(f"-> {nama} ditambah di AWAL")

    # 2. TAMBAH DI AKHIR
    def tambah_akhir(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            self.tail.next = node
```

```
            if self.double:
                node.prev = self.tail
            self.tail = node
        print(f"-> {nama} ditambah di AKHIR")

    # 3. TAMBAH DI TENGAH (URUT NIM)
    def tambah_urut_nim(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)

        if not self.head or nim < self.head.data.nim:
            self.tambah_awal(nim, nama, jurusan)
            return

        curr = self.head
        while curr.next and curr.next.data.nim < nim:
            curr = curr.next

        # Cek duplikat NIM
        if curr.data.nim == nim:
            print(f"!! NIM {nim} sudah ada!")
            return

        node.next = curr.next
        curr.next = node
        if self.double:
            if node.next:
                node.next.prev = node
            node.prev = curr
            if curr == self.tail:
                self.tail = node
        print(f"-> {nama} ditambah urut NIM")

    # 4. HAPUS BERDASARKAN NIM
    def hapus(self, nim):
        if not self.head:
            print("List kosong!")
            return

        curr = self.head
        while curr and curr.data.nim != nim:
            curr = curr.next

        if not curr:
            print(f"NIM {nim} tidak ditemukan!")
            return

        if curr == self.head:
```

```
            self.head = curr.next
            if self.head and self.double:
                self.head.prev = None
            if curr == self.tail:
                self.tail = None
        elif curr == self.tail:
            self.tail = curr.prev if self.double else self._get_prev(curr)
            if self.tail:
                self.tail.next = None
        else:
            prev_node = curr.prev if self.double else self._get_prev(curr)
            prev_node.next = curr.next
            if self.double and curr.next:
                curr.next.prev = prev_node

        print(f"-> Data NIM {nim} berhasil dihapus")

    def _get_prev(self, node_target): # Helper khusus Single Linked List
        curr = self.head
        while curr and curr.next != node_target:
            curr = curr.next
        return curr

    # 5. CARI BERDASARKAN NIM (Linear Search)
    def cari(self, nim):
        curr = self.head
        posisi = 0
        while curr:
            if curr.data.nim == nim:
                print(f"DITEMUKAN di posisi {posisi}: {curr.data.nim} |
{curr.data.nama} | {curr.data.jurusan}")
                return curr.data
            curr = curr.next
            posisi += 1
        print(f"NIM {nim} tidak ditemukan")
        return None

    # 6. TAMPILKAN SEMUA
    def tampilkan(self):
        if not self.head:
            print("Belum ada data mahasiswa")
            return
        print("\n--- DAFTAR MAHASISWA ---")
        print(f"{'NIM':<10} {'Nama':<15} {'Jurusan'}")
        print("-" * 40)
        curr = self.head
        while curr:
```

```
            print(f"{curr.data.nim:<10} {curr.data.nama:<15}
{curr.data.jurusan}")
            curr = curr.next
        print("-" * 40 + f" [Mode: {'DOUBLE' if self.double else 'SINGLE'}
Linked List]\n")

# --- PROGRAM UTAMA (MENU) ---
if __name__ == "__main__":
    print("Pilih implementasi:")
    print("1. Single Linked List")
    print("2. Double Linked List")
    pilih = input("Pilih (1/2): ")
    is_double = True if pilih == "2" else False

    ll = LinkedListMahasiswa(double=is_double)

    while True:
        print("\nMENU:")
        print("==============================")
        print("1. Tambah di Awal")
        print("2. Tambah di Akhir")
        print("3. Tambah Urut NIM (Tengah)")
        print("4. Hapus berdasarkan NIM")
        print("5. Cari berdasarkan NIM")
        print("6. Tampilkan Semua")
        print("0. Keluar")

        menu = input("Pilih menu(0-7): ")

        if menu == "0":
            break
        elif menu in ["1","2","3"]:
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            jurusan = input("Jurusan: ")
            if menu == "1": ll.tambah_awal(nim, nama, jurusan)
            elif menu == "2": ll.tambah_akhir(nim, nama, jurusan)
            else: ll.tambah_urut_nim(nim, nama, jurusan)
        elif menu == "4":
            nim = int(input("NIM yang mau dihapus: "))
            ll.hapus(nim)
        elif menu == "5":
            nim = int(input("NIM yang dicari: "))
            ll.cari(nim)
        elif menu == "6":
            ll.tampilkan()
```

Operasi	Single Linked List	Double Linked List
Tambah di awal	O(1) – langsung ubah head	O(1) – langsung ubah head
Tambah di akhir	O(1) – jika simpan tail,O(n) jika tidak	O(1) – karena tail selalu ada
Tambah urut NIM (tengah)	O(n) – harus traversal cari posisi	O(n) → traversal juga
Hapus di awal	O(1)	O(1)
Hapus di akhir	O(n) → harus cari node sebelumnya	O(1) → langsung akses prev dari tail
Hapus node tertentu	O(n) → cari node & node sebelumnya	O(n) untuk cari node, tapi update pointer O(1)
Cari berdasarkan NIM	O(n) → linear search	O(n) → linear search
Tampilkan semua data	O(n) → traversal maju	O(n) → traversal maju/mundur
Space Complexity	Hemat (1 pointer per node)	Boros (2 pointer per node)

Penjelasan

- **Insert Awal/Akhir** → konstan (O(1)) karena langsung ubah pointer head atau tail.
- **Insert Urut NIM** → linear (O(n)) karena harus mencari posisi yang tepat.
- **Delete** →
    - SLL: butuh traversal untuk menemukan node sebelumnya (O(n)).
    - DLL: lebih efisien karena ada pointer prev, jadi delete node tertentu bisa O(1) setelah ditemukan.
- **Search** → tetap O(n) karena harus cek satu per satu.
- **Space** → DLL lebih boros karena setiap node menyimpan prev dan next.

Kesimpulan

- SLL cocok untuk aplikasi sederhana dengan operasi di awal list.
- DLL lebih fleksibel untuk operasi kompleks (hapus di akhir, navigasi dua arah), tapi butuh memori ekstra.